Slide 1

CS 170

Java Programming 1

Midterm Exam 2

Duration: 00:00:42

Advance mode: Auto

Notes:

Hi Folks,

This is the CS 170, Java Programming 1 lecture on Midterm Exam 2.

All three of the CS 170 exams consist of two parts, a 50-question, closed-notes, closed book short-answer, multiple-choice exam, and a practical programming exam. This lecture will show you how to download and set up a practice programming exam that is similar to the problem you'll need to complete for the second midterm.

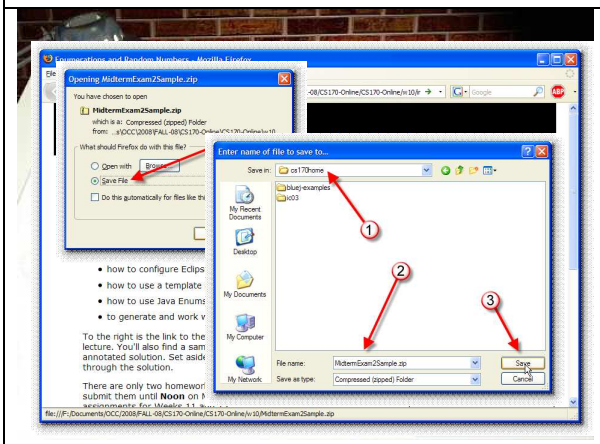
The second midterm exam will ask you to write four methods:

- one involving boolean expressions
- one involving simple loops
- one involving more complex logic problems
- one involving more complex loops

These problems are similar to those you've been doing on JavaBat, but you'll complete them using Eclipse and they'll be graded and submitted using Web-Cat. Since you have about an hour and a half to complete the programming portion of the exam, you'll want to follow the instructions in this lecture to get started, and then time yourself to see how you've done. Practicing under test conditions will really help you when it comes to the day of the test.

Once you've finished taking the practice exam, come back and watch the second-half of this lecture to double-check your work.

Ready? Let's get started.



Slide 2

Download

Duration: 00:00:51

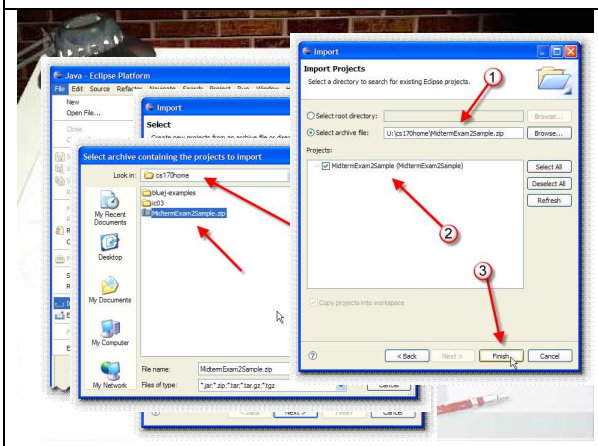
Advance mode: Auto

Notes:

Get started by downloading and opening the sample exam. The practice exam is stored as a zip file, but you don't need any "unzipping" software to open in. Instead, you'll use Eclipse to open it as a project.

To get the starter file, just click the link named MidtermExam2Sample.zip shown here.

Rather than opening the file, choose Save as shown here. Depending on how your browser is set up, you may, or may not be given an option as to where to save the file. If you're given a choice, then save it in your cs170home folder, like this. If, on the other hand, your browser saves the file on your desktop automatically, then you'll need to drag it into your cs170 folder before you go to the next step. Make sure that you don't put it in the folder containing your Eclipse workspace.



Slide 3

Setup

Duration: 00:00:59
Advance mode: Auto

Notes:

To open the exam and switch to your normal workspace. Then, choose Import from the Eclipse file menu as shown in the slide here.

When the "Import Select" dialog box appears as shown here:

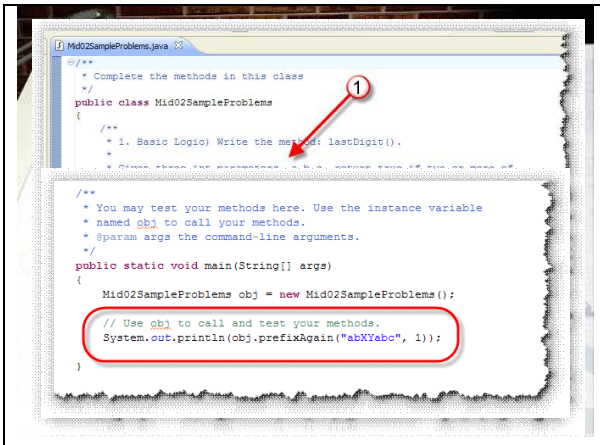
- expand the General folder as shown at (1.
- select the "Existing Projects into Workspace" in the sub-list at (2.
- and then, click the Next button at (3.

When the "Import Projects" dialog appears you should click the radio button that says "Select Archive File" and then click the "Browse" button appearing to its right.

Browse to the location where you saved the MidtermExam2Sample.zip file (cs170home in my case), select it, and click the Open button.

When the "Import Projects" dialog reappears for the second time, it will have the archive file name selected (at 1), and the MidtermExam2Sample project checked in the center section of the screen (marked 2 in my slide here). This is the project that will be imported. Note that you cannot import this if you already have a project with this name. In that case, you'll need to rename the existing project (using File->Refactor) before importing the new project.

When you've got everything looking like this, click Finish, and the Sample Midterm project will be imported into your workspace.



Slide 4

Unit Tests

Duration: 00:01:18
Advance mode: Auto

Notes:

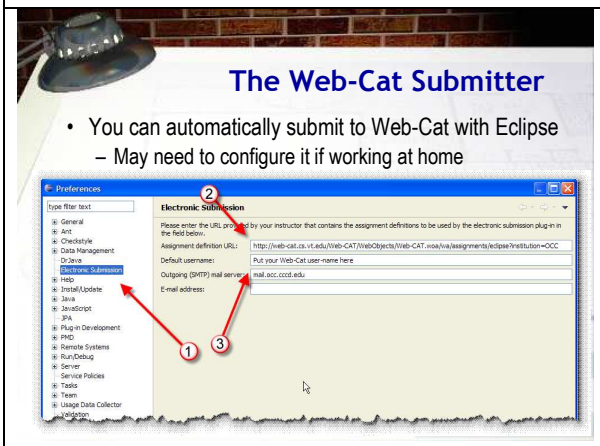
When Eclipse imports your project into the workspace, you need to expand the default package (shown as number 1) here and then double-click the source-code file that it contains to open the program in the editor.

All four problems are contained in a single class. For each problem I've provided:

1) a brief description including the name.
2) a few sample results that you can use to test your code. The actual Web-cat tests are much more extensive.
3) the JavaDoc for each of the parameters and for the returned value.

Underneath the documentation for each of the four problems, you'll write the requested method. In this case, you'd write a method named lastDigit that returns true or false and that has three int parameters.

Unlike the first exam, I have not supplied JUnit tests for these problems. You'll need to write your own tests in the main() method to make sure your code does what you expect. Inside the main() method, I have already created an instance of the class containing your midterm code, named obj. To see the results of each of your methods, you can use System.out.println() to print the results of calling your methods. In the example shown here, I'm calling the prefixAgain() method, and I'll compare the results that are printed to the results listed in the problem statement documentation.



Slide 5

The Web-Cat Submitter

Duration: 00:00:30
Advance mode: Auto

Notes:

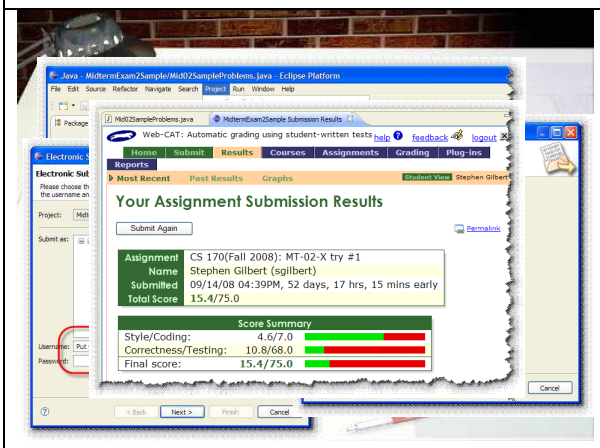
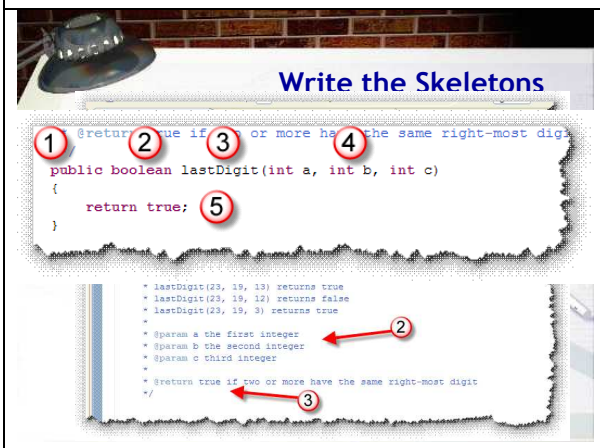
To submit your exam for grading you'll use Web-Cat. Eclipse comes with a submitter extension (just like BlueJ) that lets you submit your assignments from within the IDE. In the Computing Center, this should already be set up and ready to go. If you're working at home, though, you'll probably need to complete the Electronic Submission properties sheet. Here's how to do that.

First, open the Preferences dialog. On Windows and Linux that will be on the Window menu. On the Mac, it should be on the Eclipse menu following the OSX conventions.

When the preferences dialog appears, you'll need to first select the Electronic Submission tab. The most important part of this dialog is the URL to submit your assignments to. That URL is: http://web-cat.cs.vt.edu/Web-CAT/WebObjects/Web-CAT.woa/wa/assignments/eclipse?institution=OCC

Of course, that's a really long and easy to mess up URL, so I've added a link to a text file on this week's class Web page. Just click that and copy and paste it in location # 2.

For number 3, you can put in your Web-Cat user id, or just

	leave it blank. If you leave it blank, then you'll have to enter it each time you submit an assignment. In either case, each time you submit an assignment, you'll need to enter your Web-Cat password. The outgoing mail server field isn't really used with the Web-Cat submitter, but you can fill it in with mail.occ.cccd.edu. You can also fill in your own email address if you like.
 **Slide 6** **Submission** Duration: 00:01:34 Advance mode: Auto	## Notes: Once you've completed all four problems (or the time is called on the exam), you'll turn in your exam using Web-Cat and the Eclipse submitter. The Eclipse submitter works pretty much like the BlueJ Submitter you've already used. • On the Project menu, simply choose Submit Project • When the Electronic Submission dialog appears, select the MT-02-X project (marked #1 in the slide here) and then enter your Web-Cat ID and password for number 2. Click the Next button when you've done that. If you typed your Web-cat username and password correctly, you'll see a "Success" dialog that looks like this. Click the Finish button and your Web-Cat page will open up in Eclipse and you can check your result. As you can see from the example here, I've just gotten started on solving the problems in the exam. Now, before I go ahead and walk through the solution, why don't you close this presentation and try taking the exam yourself. Minimize distractions and limit yourself to an hour and a half. When you're done, check your work against the solution shown in the second half of this presentation.
 **Slide 7** **Write the Skeletons** Duration: 00:01:16 Advance mode: Auto	## Notes: All finished? OK, now let's walk through the solution. The most effective way to solve this kind of problem is to work on the mechanics or method skeletons before you even think about the logic at all. That also ensures that you won't get a 0. Let me show you what I mean. Locate the first method in the set of problems. Notice that: 1) its name is lastDigit 2) it requires three int parameters 3) it returns a true-false value Using just that information, you can write a method skeleton that will satisfy Web-Cat as far as the structural requirements of the method go. Here's how you write that skeleton: 1) always start with the keyword public. I can't test your

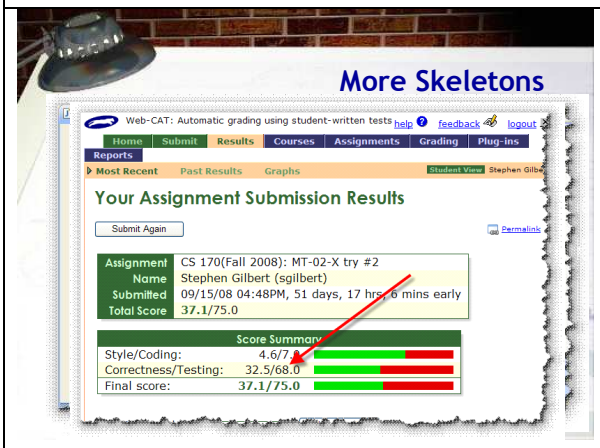
methods unless they are public.

2) because lastDigit returns a true-false value, the return-type is specified as boolean (with a lowercase b).

3) the name of the method comes next. Make sure you spell it exactly as it is specified in the problem statement, including all capitalization. The name of the method is followed by a set of parentheses, but NO semicolon.

4) inside the semicolon you'll place the parameters. For every param specified in the problem's JavaDoc, you'll create one variable. This problem has three params, the ints a, b, and c.

5) following the method body, you'll add some braces. If the return type is void, then you're done at this point. If the return type is something other than void, you need to add a return expression. For a boolean method, return true, for a numeric method, return 0 and for a String method, return the empty String, just to get everything syntactically correct.



Slide 8

More Skeletons

Duration: 00:00:32
Advance mode: Auto

Notes:

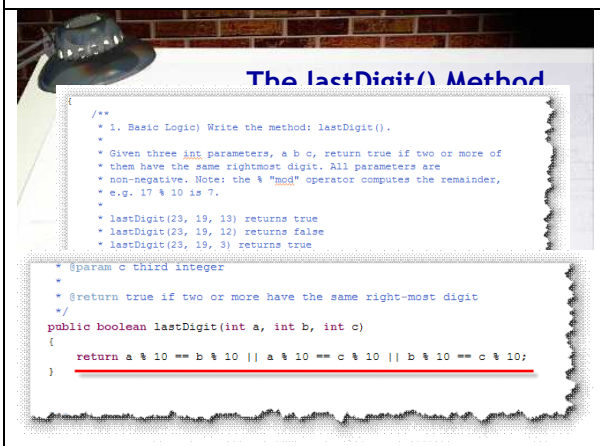
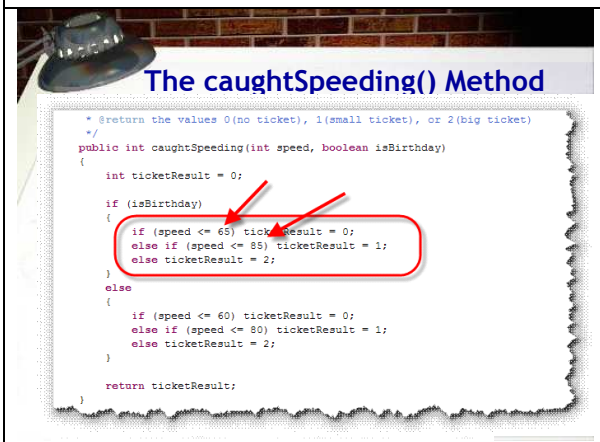
Now let's look at the rest of the methods, starting with Problem 2, the caughtSpeeding() method.

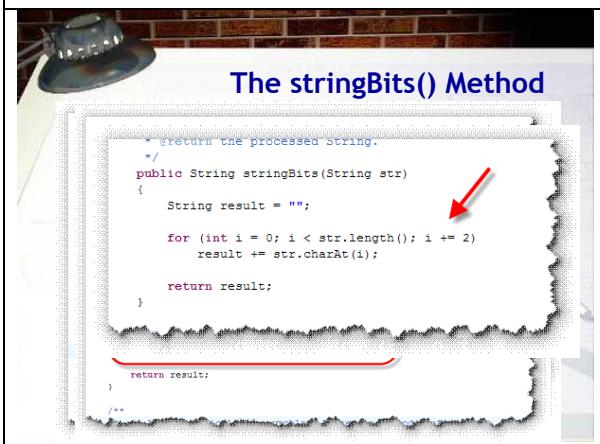
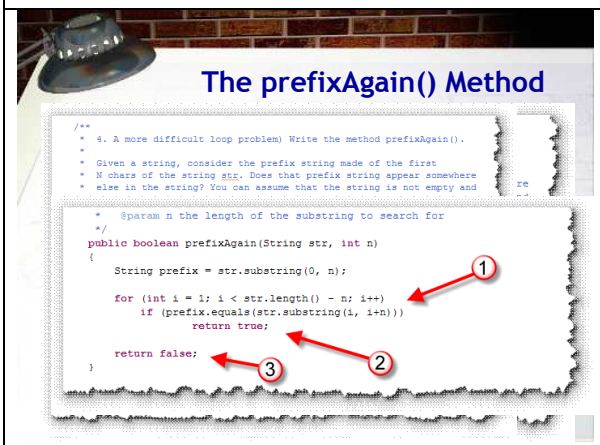
- Like all methods, the skeleton starts with the keyword public, followed by the return type. In this case, if you look at the JavaDoc section marked 3), you'll see that the return values are 0, 1 or 2 so we put int as the return type.
- The next part of the heading is the name of the method, which we pick up from the first line in the JavaDoc (marked 1 in my slide).
- Next we have the parameters. As you can see from the JavaDoc (marked 2), there are two parameters, speed and isBirthday. If you look at the three examples, you can see that speed is an int and isBirthday is a boolean.
- Finally, we have the method body, enclosed in braces. Since caughtSpeeding is a non-void method, a return expression is required. For right now, we'll just return 0, which is an int, so it matches the method return type.

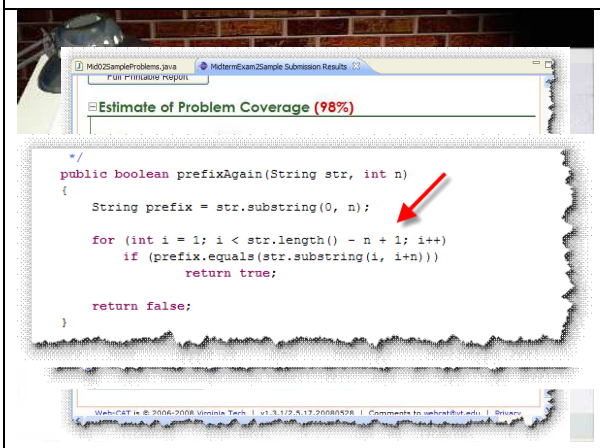
The third method is stringBits(). This method returns a String, so the header starts with public String. It also takes a single String parameter. Since the method returns a String, the return statement (marked 3 in the slide) just returns the empty String.

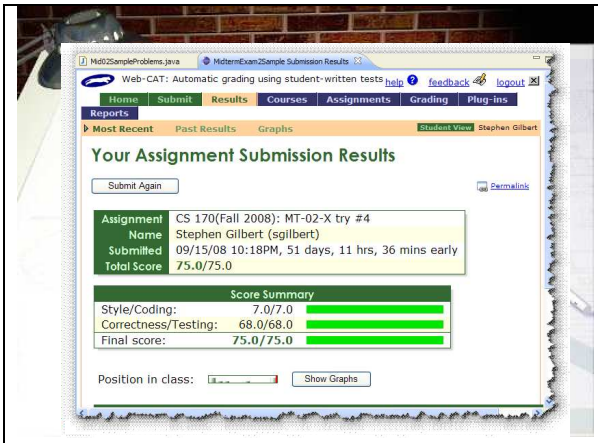
The last method in the sample midterm is prefixAgain(). This is a boolean method. (Even though I forgot to add JavaDoc @return statement, you can tell it's a boolean method by the samples marked 1.)

The prefixAgain() method has two parameters, a String and an int. Since it's a boolean method, the return expression just

	returns true. Once you've done this, the basic structure of your program is done. You can do this part just by remembering how to define a method. If we submit the program to Web-Cat at this point, you'll see that this is worth up to 50% of the exam points.
 **Slide 9** **The lastDigit() Method** Duration: 00:02:08 Advance mode: Auto	**Notes:** Now, let's start with the coding. I'll start on the first method, lastDigit(). Note that the method returns true if two or more of the three int parameters, a, b, or c, have the same rightmost digit. Also, note that we can find the rightmost digit of any integer by using the expression n "mod" 10 where "mod" is Java's % sign remainder operator. So, to start out, let's just return true if a and b have the same rightmost digit. The rightmost digit of a is the expression a % 10 and the rightmost digit of b is b % 10. To compare them we use the equality operator ==. So, the return expression shown here will return true if a and b have the same rightmost digit and false otherwise. What we really want, though is to return true if any two (or more) pairs have the same rightmost digit. So, how many pairs are there? Three: ab, ac, and bc since the order doesn't matter (that is ab and ba are the same for this problem). So, to see if it's true for any of those three pairs, we use two more copies of the same kind of Boolean expression (with a and b replaced appropriately), combined with the OR operator like this.
 **Slide 10** **The caughtSpeeding() Method** Duration: 00:00:25 Advance mode: Auto	**Notes:** The caughtSpeeding() method is also a logic problem, but one that is easier solved with if and else. The only question is which form of if and else. Well, notice that the parameters are two types: speed and whether or not it is your birthday. This is really similar to the income tax problem we looked at in class, where you first decide on the filing status and then make your calculations based on the income brackets. When you have leveled decisions like this, usually the easiest way to write them is to use nested if statements. The first level of nesting concerns your birthday. Write an if-else statement using isBirthday. Put the birthday speed brackets (which are 5 mph faster) in the true block and the regular speed brackets in the false block like this. Note that when you test a Boolean condition, you always just use the variable itself to calculate the true or false value. You should never write if (isBirthday == true) because it is so error-prone. Since the regular case is inside the else portion, let's write that

	first. Before we do, though, let's add a variable to hold the ticketResult and modify the return statement to return that value. I've marked those 1 and 2 in the slide here. To calculate the ticket's value, we're use sequential if-else-if statements. The first one handles values less-that-or equal to 60, the second handles values from 61 through 80 while the final else handles everything greater than 80. Finally, since the isBirthday values are just 5 miles-per-hour faster, you can copy the entire block into the if (isBirthday) block and modify the 60 to 65 and the 80 to 85.
 **Slide 11** **The stringBits() Method** Duration: 00:00:25 Advance mode: Auto	**Notes:** stringBits() is the simple loop problem on the example program. This program wants you to take an input String and copy every other character to an output String, which is returned. To start out, first add an output variable, result, and initialize it to the empty String. Then, modify the existing return statement so that it returns result, instead of the existing empty string literal. Now, to process the input String, we'll use a loop. Since we want to go around a "known" number of times, we can use a counter-controlled loop, using the String length as the loop bounds. As you learned in class, the best loop in this situation is a for loop. Write a for loop that goes through every character, and then use charAt() to extract the current character and concatenation to paste it onto the end of the output String like this. Well, that looks fine, except, it copies every character from the input String to the output String. To copy every other character, you could test if the variable i was even or odd and only assign the value to the output when it was odd. An easier way, though, is just to use the for loop's update expression to copy every other character, like this.
 **Slide 12** **The prefixAgain() Method**	**Notes:** The most complex loop method is prefixAgain() which returns true if the first n characters of the String passed as a parameter appear again inside the String. To start this off, you can extract the prefix by using the substring() method. The method documentation guarantees that n will be between 1 and the String length so you don't have to handle the empty String. Now, let's write our loop. We don't want to start looking at position 0 (since it will obviously find a match; after all, that's where we got the substring from to begin with). We'll start at position 1 in case the word appears again overlapping part of the prefix. We also don't want to go all the way to the end of

Duration: 00:00:25 Advance mode: Auto	the String, because then, when we try to compare the character in the last position, the prefix String will extend beyond the end of the String parameter, causing a runtime error. Instead we need to subtract n from the length of the parameter, like this. (Note that I've used just a semicolon for the loop body to get it to compile.) To compare the prefix and the current position of the String, we should use the equals() method, along with the substring for the n characters starting a the index value i. Use the condition marked 1 inside an if statement. If the statement is true, then return true. If you go through the entire remaining portion of the String and haven't found a match, then you should return false as the last statement in the method. (Marked 3 in the slide shown here.)
 **Slide 13** **Check the Result** Duration: 00:00:25 Advance mode: Auto	**Notes:** When you submit this version to Web-Cat you can see it's much closer to being finished. We lost only a point and a half to correctness and four points to style. Let's handle the correctness problems first. Scroll on down to the Estimate of Problem coverage. As you can see, the program is 98% correct. The only problem is with the prefixAgain method with the values aa and 1. Since the single character prefix "a" does appear again in the String "aa", that means the method should return true. Since it's showing up here as an error, you know that our version returns false in this case. Let's look at the code and see why. The input String is "aa", so the prefix contains the substring starting at 0 and going up-to, but not including position 1. In other words, only the character at position 0 or "a". That looks correct. So let's look at the loop. The loop starts at position 1 (the second "a") and goes while i is less than str.length() – n (or 1 in this case). Since str.length() is 2 and n is 1, we'll continue while i is less than 1. Of course, that means that we'll never enter the loop, since the loop starts at 1 which is never less than 1. You might be tempted, at this point, to simply remove the – n after the length() method. While that would fix this particular case, it would cause many of the other tests to fail. You should recognize this problem as a "fencepost", or "off-by-one" error. The correct solution is just to add 1 to the condition, like this.



Slide 14

Style Errors

Duration: 00:00:32
Advance mode: Auto

Notes:

Before submitting back to Web-Cat, let's see what the style problems are, since those should be four quick points. Locate and click the hyperlink under Files as shown here.

When you open the file, you'll see three errors that have to do with JavaDocs. You'll need to complete the JavaDoc at the top of the file with your own name and the date. Also, you should add a period to the first sentence.

Scrolling down, you'll find the other errors having to do with spacing around the plus operator. Fix that and resubmit to Web-Cat.

If all goes well, you should see the all-green of a perfect score.